

Reviewer Pack — Layer-Commutator Frobenius Discrepancy Separates Read-Twice ...

Entry 0e06b5f9fc52 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-16 20:13:48 UTC

Layer-Commutator Frobenius Discrepancy Separates Read-Twice BP for IP_2

Entry ID: 0e06b5f9fc52 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-16 20:13:48 UTC

1. Conjecture

Field A (mathematical branch): Non-commutative operator discrepancy (Frobenius norm of layer-transition commutators, viewed as a Σ^b_1 -witnessing measure in bounded arithmetic à la Cook-Krajíček) **Field B** (complexity object): Read-twice branching programs computing the inner-product-mod-2 function IP_2

Statement:

Let P be a read-twice branching program with state set S ($|S|=s$) on n Boolean variables. For each of the $2n$ read-positions $r \in \{1, \dots, 2n\}$ let $A_r \in R^{\{s \times s\}}$ be the averaged transition matrix $A_r := (T_r^0 + T_r^1)/2$, where T_r^b is the 0/1 matrix sending state u to state v if reading bit b at position r drives $u \rightarrow v$. Define the layer-commutator Frobenius discrepancy $\rho(P) := \log_2(1 + \sum_{1 \leq r < q \leq 2n} \|A_r A_q - A_q A_r\|_F^2 / s^2)$. Conjecture: (a) $\rho(P) \leq 6 \cdot \log_2(s) + 4$ for every read-twice BP P ; (b) for every read-twice BP P that computes IP_2 on $n \geq 4$ inputs, $\rho(P) \geq (1/8) \cdot n$. A single P violating either inequality refutes the conjecture; together they would force $\text{size}(P) \geq 2^{\{n/48\}}$ for read-twice BPs computing IP_2 .

Rationale (proposer's reasoning):

Bounded-arithmetic witnessing of BP membership relies on local invariants that are Σ^b_1 -definable from the BP layer data; commutator energies of averaged layer matrices are exactly that kind of computable witness, while operator non-commutativity is the algebraic signature of the bilinear structure $IP_2(x,y) = \langle x,y \rangle_2$. Read-once BPs decompose into block-triangular layers where averaged transitions essentially commute on each reachable mass, but IP_2 forces every read-twice realization to mix x -reads and y -reads non-trivially. Because ρ is a Frobenius (L^2 spectral) quantity rather than a polynomial in the entries, the construction does not algebrize and the invariant is computed on BP structure (not the truth table), so it dodges Natural Proofs.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a3cc03eb4e88f9bd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{4, 6, 8, 10, 12\}$ with 30 seeds each (150 trials), for every read-twice BP sampled compute $\rho(P)/\log_2(s)$; for every BP computing IP 2 compute $\rho(P)/n$. Support requires both ratio bounds hold on 100% of trials; one violation falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - read-twice branching program inner product mod 2 lower bound - noncommutative discrepancy Frobenius commutator branching program - transition matrix commutator lower bound IP2 bounded arithmetic

Top relevant hits considered: - [<http://arxiv.org/abs/1709.08890v2>] Partial matching width and its application to lower bounds for branching programs - [<http://arxiv.org/abs/1007.1875v2>] Lower Bounds for Quantum Oblivious Transfer - [<http://arxiv.org/abs/0710.5926v2>] Mod 2 cohomology of 2-local finite groups of low rank - [<http://arxiv.org/abs/2603.27466v1>] English translation of Frobenius' and Stickelberger's "On the theory of elliptic functions" - [<http://arxiv.org/abs/0906.1662v2>] Frobenius morphisms of noncommutative blowups - [<http://arxiv.org/abs/0705.1265v2>] A noncommutative Bohnenblust-Spitzer identity for Rota-Baxter algebras solves Bogoliubov's recursion - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/1210.1461v1>] A Scalable CUR Matrix Decomposition Algorithm: Lower Time Complexity and Tighter Bound

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from fractions import Fraction

def matrix_mult(A, B):
    """Multiply two matrices A and B."""
    if not A or not B:
        return []
    rows_A = len(A)
```

```

cols_A = len(A[0]) if rows_A > 0 else 0
rows_B = len(B)
cols_B = len(B[0]) if rows_B > 0 else 0

if cols_A != rows_B:
    return []

result = [[0 for _ in range(cols_B)] for _ in range(rows_A)]
for i in range(rows_A):
    for j in range(cols_B):
        for k in range(cols_A):
            result[i][j] += A[i][k] * B[k][j]
return result

def matrix_sub(A, B):
    """Subtract matrix B from matrix A."""
    if len(A) != len(B) or len(A[0]) != len(B[0]):
        return []
    return [[A[i][j] - B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def matrix_frobenius_norm(A):
    """Compute the Frobenius norm of matrix A."""
    if not A:
        return 0.0
    return math.sqrt(sum(sum(a * a for a in row) for row in A))

def build_brs_bp(n):
    """Build the BRS read-once BP for IP_2."""
    s = 2 ** (n // 2)
    states = list(range(s))
    transitions = []
    for r in range(2 * n):
        if r < n:
            T0 = [[0] * s for _ in range(s)]
            T1 = [[0] * s for _ in range(s)]
            for u in states:
                v = u
                T0[u][v] = 1
                T1[u][v] = 1
            transitions.append((T0, T1))
        else:
            T0 = [[0] * s for _ in range(s)]
            T1 = [[0] * s for _ in range(s)]
            for u in states:
                v = u ^ (1 << (r - n))
                T0[u][v] = 1
                T1[u][v] = 1
            transitions.append((T0, T1))
    return states, transitions

def build_interleaved_parity_bp(n):
    """Build the interleaved-parity read-twice BP for IP_2."""
    states = [0, 1]
    transitions = []
    for r in range(2 * n):
        if r % 2 == 0:

```

```

        T0 = [[1, 0], [0, 1]]
        T1 = [[1, 0], [0, 1]]
    else:
        T0 = [[1, 0], [0, 1]]
        T1 = [[0, 1], [1, 0]]
    transitions.append((T0, T1))
    return states, transitions

def build_random_bp(n, w):
    """Build a random read-twice BP of width w."""
    s = w
    states = list(range(s))
    transitions = []
    for r in range(2 * n):
        T0 = [[random.randint(0, 1) for _ in range(s)] for _ in range(s)]
        T1 = [[random.randint(0, 1) for _ in range(s)] for _ in range(s)]
        transitions.append((T0, T1))
    return states, transitions

def compute_rho(P, n):
    """Compute the layer-commutator Frobenius discrepancy  $\rho(P)$ ."""
    states, transitions = P
    s = len(states)
    rho = 0.0
    for r in range(2 * n):
        for q in range(r + 1, 2 * n):
            T0_r, T1_r = transitions[r]
            T0_q, T1_q = transitions[q]
            A_r = [(T0_r[i][j] + T1_r[i][j]) / 2 for j in range(s)] for i in range(s)]
            A_q = [(T0_q[i][j] + T1_q[i][j]) / 2 for j in range(s)] for i in range(s)]
            A_rA_q = matrix_mult(A_r, A_q)
            A_qA_r = matrix_mult(A_q, A_r)
            diff = matrix_sub(A_rA_q, A_qA_r)
            norm = matrix_frobenius_norm(diff)
            rho += norm ** 2
    rho = math.log2(1 + rho / (s ** 2))
    return rho

def evaluate_bp(P, n):
    """Evaluate the BP on all possible inputs."""
    states, transitions = P
    s = len(states)
    current_state = 0
    for r in range(2 * n):
        T0, T1 = transitions[r]
        if r < n:
            bit = 0
        else:
            bit = 1
        next_state = 0
        for i in range(s):
            if T0[current_state][i] == 1 and bit == 0:
                next_state = i
                break
            if T1[current_state][i] == 1 and bit == 1:
                next_state = i

```

```

        break
    current_state = next_state
return current_state

def run_trial(seed):
    random.seed(seed)
    n_values = [4, 6, 8, 10, 12]
    metric_values = []
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        # Build the BRS BP
        brs_bp = build_brs_bp(n)
        rho_brs = compute_rho(brs_bp, n)
        s_brs = len(brs_bp[0])
        ratio_brs = rho_brs / math.log2(s_brs)
        if ratio_brs > 6:
            conjecture_holds = False
            counterexample = f"BRS BP with n={n}, s={s_brs}, rho={rho_brs}, ratio={ratio_brs}"

        # Build the interleaved-parity BP
        interleaved_bp = build_interleaved_parity_bp(n)
        rho_interleaved = compute_rho(interleaved_bp, n)
        s_interleaved = len(interleaved_bp[0])
        ratio_interleaved = rho_interleaved / math.log2(s_interleaved)
        if ratio_interleaved > 6:
            conjecture_holds = False
            counterexample = f"Interleaved-parity BP with n={n}, s={s_interleaved}, rho={rho_interleaved}, ratio={ratio_interleaved}"

        # Build random BPs
        for w in [2, 4, 8, 16]:
            for _ in range(30):
                random_bp = build_random_bp(n, w)
                rho_random = compute_rho(random_bp, n)
                s_random = len(random_bp[0])
                ratio_random = rho_random / math.log2(s_random)
                if ratio_random > 6:
                    conjecture_holds = False
                    counterexample = f"Random BP with n={n}, s={s_random}, rho={rho_random}, ratio={ratio_random}"

                # Check if the BP computes IP_2
                if evaluate_bp(random_bp, n) == 1:
                    ratio_ip2 = rho_random / n
                    if ratio_ip2 < 1/8:
                        conjecture_holds = False
                        counterexample = f"Random BP computing IP_2 with n={n}, s={s_random}, rho={rho_random}, ratio={ratio_ip2}"

            metric_values.append(ratio_random)
            instances_tested += 1

    if not metric_values:
        return {
            "metric_name": "rho_ratio",
            "metric_value": 0.0,

```

```

        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No instances tested"
    }

    mean_metric = sum(metric_values) / len(metric_values)
    return {
        "metric_name": "rho_ratio",
        "metric_value": mean_metric,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
    metric_values = []
    conjecture_holds_all = True
    counterexample = ""

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        metric_values.append(result["metric_value"])
        if not result["conjecture_holds"]:
            conjecture_holds_all = False
            counterexample = result["counterexample"]

    if not metric_values:
        print("RESULT: INCONCLUSIVE reason=no_metric_values")
        sys.exit(0)

    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if conjecture_holds_all:
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    else:
        print(f"RESULT: FALSIFIED counterexample={counterexample} first_failing_seed={seeds[0]}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_12f9b311.py", line 211, in <module>

```

    result = run_trial(seed)
    ~~~~~
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_12f9b311.py", line 148, in run_trial

```

    brs_bp = build_brs_bp(n)
    ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_12f9b311.py", line 67, in build_brs_bp
    T0[u][v] = 1
    ~~~~~^^^
```

IndexError: list assignment index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError in build_brs_bp before any trials produced data, so the pre-registered support condition cannot be evaluated. | next: Fix the indexing bug in build_brs_bp (state-set sizing for the BRS construction) and rerun the 150-trial sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	123303
2	preregistration	claude_max	opus	0	0	7247
3	novelty	claude_max	opus	0	0	3651
4	novelty	claude_max	opus	0	0	7106
5	test_gen	mistral	codestral-latest	0	0	317363
6	test_gen	mistral	codestral-latest	0	0	313376
7	test_gen	mistral	codestral-latest	0	0	318332
8	test_gen	mistral	codestral-latest	0	0	316916
9	judge	claude_max	opus	0	0	3816

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1411109 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/0e06b5f9fc52.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0e06b5f9fc52.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0e06b5f9fc52.tar.gz` (if generated)